

DSC 190/291 Assignment 7

Detailed Write-Up

Dhruv Patel

May 27, 2026

Contents

1	Kernel Methods	2
2	ℓ_1 Rademacher Complexity	11
3	Normalized ℓ_1 Margins and FixedBoost	23
4	FixedBoost Margin Experiment	40
5	Agent Skill Reflection	55

1. Kernel Methods

This homework continues the scale-sensitive complexity theme from Homework 6, where generalization is controlled by norms and margins rather than by dimension.

Let $\mathcal{U}$ be a Hilbert space with inner product $\langle \cdot, \cdot \rangle$ and induced norm $\| \cdot \|$, and let $\varphi : \mathcal{X} \rightarrow \mathcal{U}$ be a feature map with associated kernel

$$K(x, x') = \langle \varphi(x), \varphi(x') \rangle.$$

For $B > 0$, let

$$\mathcal{H}_{K,B} = \{ x \mapsto \langle w, \varphi(x) \rangle : w \in \mathcal{U}, \|w\| \leq B \}$$

be the predictors of RKHS norm at most B .

For a sample $S = (x_1, \dots, x_n)$ and a real-valued function class $\mathcal{F}$, write the empirical Rademacher complexity

$$\hat{\mathcal{R}}_S(\mathcal{F}) = \mathbb{E}_\sigma \sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i),$$

where $\sigma_1, \dots, \sigma_n$ are independent uniform signs. Write also

$$\mathcal{R}_{\mathcal{D},n}(\mathcal{F}) = \mathbb{E}_{S \sim \mathcal{D}^n} \hat{\mathcal{R}}_S(\mathcal{F}).$$

1. Norm-based kernel bound.

Prove the empirical bound

$$\hat{\mathcal{R}}_S(\mathcal{H}_{K,B}) \leq \frac{B}{n} \sqrt{\sum_{i=1}^n K(x_i, x_i)},$$

and conclude the distribution-level bound

$$\mathcal{R}_{\mathcal{D},n}(\mathcal{H}_{K,B}) \leq B \sqrt{\frac{\mathbb{E}[K(X, X)]}{n}}.$$

Goal.

We want to derive an upper bound on the empirical Rademacher complexity of the RKHS ball

$$\mathcal{H}_{K,B} = \{ x \mapsto \langle w, \varphi(x) \rangle : \|w\| \leq B \},$$

where the complexity bound depends only on the norm bound B and the kernel diagonal terms

$$K(x_i, x_i).$$

The important idea is that although the feature space $\mathcal{U}$ may be extremely high-dimensional or even infinite-dimensional, the complexity of the class can still be controlled entirely through kernel evaluations.

Step 1: Start from the empirical Rademacher complexity definition.

Recall that the empirical Rademacher complexity of a function class $\mathcal{F}$ on a sample

$$S = (x_1, \dots, x_n)$$

is

$$\hat{\mathcal{R}}_S(\mathcal{F}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right],$$

where

$$\sigma_1, \dots, \sigma_n$$

are independent Rademacher random variables taking values in $\{-1, +1\}$ uniformly. For the RKHS ball $\mathcal{H}_{K,B}$, every predictor has the form

$$f_w(x) = \langle w, \varphi(x) \rangle$$

with

$$\|w\| \leq B.$$

Substituting this into the definition gives

$$\hat{\mathcal{R}}_S(\mathcal{H}_{K,B}) = \mathbb{E}_\sigma \sup_{\|w\| \leq B} \frac{1}{n} \sum_{i=1}^n \sigma_i \langle w, \varphi(x_i) \rangle.$$

Using linearity of the inner product, move w outside the sum:

$$= \mathbb{E}_\sigma \sup_{\|w\| \leq B} \frac{1}{n} \left\langle w, \sum_{i=1}^n \sigma_i \varphi(x_i) \right\rangle.$$

At this point, the entire optimization problem has been reduced to understanding the geometry of the random vector

$$\sum_{i=1}^n \sigma_i \varphi(x_i)$$

inside the Hilbert space.

Step 2: Apply the Cauchy–Schwarz inequality.

For any vectors u, v in a Hilbert space,

$$\langle u, v \rangle \leq \|u\| \|v\|.$$

Applying this inequality with

$$u = w, \quad v = \sum_{i=1}^n \sigma_i \varphi(x_i),$$

we obtain

$$\left\langle w, \sum_{i=1}^n \sigma_i \varphi(x_i) \right\rangle \leq \|w\| \left\| \sum_{i=1}^n \sigma_i \varphi(x_i) \right\|.$$

Since the supremum is only over vectors satisfying

$$\|w\| \leq B,$$

the largest possible value of the inner product is bounded by

$$B \left\| \sum_{i=1}^n \sigma_i \varphi(x_i) \right\|.$$

Therefore,

$$\hat{\mathcal{R}}_S(\mathcal{H}_{K,B}) \leq \frac{B}{n} \mathbb{E}_\sigma \left\| \sum_{i=1}^n \sigma_i \varphi(x_i) \right\|.$$

So far, we have reduced the problem to controlling the expected norm of a random signed combination of feature vectors.

Step 3: Use Jensen's inequality to simplify the expectation.

The square-root function is concave, so Jensen's inequality implies

$$\mathbb{E}[Z] \leq \sqrt{\mathbb{E}[Z^2]}$$

for any nonnegative random variable Z .

Apply this with

$$Z = \left\| \sum_{i=1}^n \sigma_i \varphi(x_i) \right\|.$$

Then

$$\mathbb{E}_\sigma \left\| \sum_{i=1}^n \sigma_i \varphi(x_i) \right\| \leq \sqrt{\mathbb{E}_\sigma \left\| \sum_{i=1}^n \sigma_i \varphi(x_i) \right\|^2}.$$

Substituting into the previous inequality gives

$$\hat{\mathcal{R}}_S(\mathcal{H}_{K,B}) \leq \frac{B}{n} \sqrt{\mathbb{E}_\sigma \left\| \sum_{i=1}^n \sigma_i \varphi(x_i) \right\|^2}.$$

Now the remaining task is to compute this second moment exactly.

Step 4: Expand the squared norm.

Using the Hilbert space identity

$$\|v\|^2 = \langle v, v \rangle,$$

we expand:

$$\left\| \sum_{i=1}^n \sigma_i \varphi(x_i) \right\|^2 = \left\langle \sum_{i=1}^n \sigma_i \varphi(x_i), \sum_{j=1}^n \sigma_j \varphi(x_j) \right\rangle.$$

Using bilinearity of the inner product,

$$= \sum_{i=1}^n \sum_{j=1}^n \sigma_i \sigma_j \langle \varphi(x_i), \varphi(x_j) \rangle.$$

Since

$$\langle \varphi(x_i), \varphi(x_j) \rangle = K(x_i, x_j),$$

this becomes

$$= \sum_{i=1}^n \sum_{j=1}^n \sigma_i \sigma_j K(x_i, x_j).$$

Take expectation over the independent Rademacher signs.

For $i \neq j$,

$$\mathbb{E}[\sigma_i \sigma_j] = \mathbb{E}[\sigma_i] \mathbb{E}[\sigma_j] = 0,$$

because each Rademacher variable has mean zero.

For diagonal terms,

$$\mathbb{E}[\sigma_i^2] = 1.$$

Therefore every off-diagonal term vanishes, leaving only the diagonal contributions:

$$\mathbb{E}_\sigma \left\| \sum_{i=1}^n \sigma_i \varphi(x_i) \right\|^2 = \sum_{i=1}^n K(x_i, x_i).$$

Substitute this back into the previous inequality:

$$\widehat{\mathcal{R}}_S(\mathcal{H}_{K,B}) \leq \frac{B}{n} \sqrt{\sum_{i=1}^n K(x_i, x_i)}.$$

Thus we obtain the empirical bound

$$\widehat{\mathcal{R}}_S(\mathcal{H}_{K,B}) \leq \frac{B}{n} \sqrt{\sum_{i=1}^n K(x_i, x_i)}.$$

Step 5: Derive the distribution-level bound.

Now take expectation over random samples

$$S \sim \mathcal{D}^n.$$

By definition,

$$\mathcal{R}_{\mathcal{D}^n}(\mathcal{H}_{K,B}) = \mathbb{E}_S \widehat{\mathcal{R}}_S(\mathcal{H}_{K,B}).$$

Using the empirical bound,

$$\mathcal{R}_{\mathcal{D}^n}(\mathcal{H}_{K,B}) \leq \mathbb{E}_S \left[\frac{B}{n} \sqrt{\sum_{i=1}^n K(x_i, x_i)} \right].$$

Apply Jensen's inequality again:

$$\mathbb{E}[\sqrt{Z}] \leq \sqrt{\mathbb{E}[Z]}.$$

Therefore,

$$\mathcal{R}_{\mathcal{D}^n}(\mathcal{H}_{K,B}) \leq \frac{B}{n} \sqrt{\mathbb{E}_S \sum_{i=1}^n K(x_i, x_i)}.$$

Since the examples are i.i.d.,

$$\mathbb{E}_S \sum_{i=1}^n K(x_i, x_i) = n\mathbb{E}[K(X, X)].$$

Substituting gives

$$\mathcal{R}_{\mathcal{D}^n}(\mathcal{H}_{K,B}) \leq \frac{B}{n} \sqrt{n\mathbb{E}[K(X, X)]}.$$

Simplifying,

$$\boxed{\mathcal{R}_{\mathcal{D}^n}(\mathcal{H}_{K,B}) \leq B \sqrt{\frac{\mathbb{E}[K(X, X)]}{n}}}.$$

Interpretation.

This result is important because it shows that the complexity of the RKHS hypothesis class depends on:

- the norm constraint B ,
- the average kernel diagonal magnitude,
- the sample size n ,

but does *not* explicitly depend on the ambient dimension of the feature space.

Even if the feature map

$\varphi(x)$ lives in an infinite-dimensional Hilbert space, the class can still have controlled complexity as long as the RKHS norm and kernel diagonal terms remain bounded.

2. Representer theorem.

The space $\mathcal{U}$ may be infinite dimensional, yet for any $\lambda > 0$ the regularized objective

$$J(w) = \frac{1}{n} \sum_{i=1}^n \text{loss}(\langle w, \varphi(x_i) \rangle; y_i) + \lambda \|w\|^2$$

can be minimized over finitely many coefficients. This is the representer theorem: prove that whenever J attains a minimum over $w \in \mathcal{U}$, some minimizer has the form

$$w^* = \sum_{j=1}^n \alpha_j \varphi(x_j), \quad \alpha \in \mathbb{R}^n.$$

Then rewrite J as a minimization over $\alpha \in \mathbb{R}^n$ in which the data enter only through the Gram matrix K_S , with entries

$$(K_S)_{ij} = K(x_i, x_j).$$

Goal.

We want to prove the representer theorem. The key statement is that even though the Hilbert space

$$\mathcal{U}$$

may be infinite-dimensional, the optimization problem

$$J(w) = \frac{1}{n} \sum_{i=1}^n \text{loss}(\langle w, \varphi(x_i) \rangle; y_i) + \lambda \|w\|^2$$

can always be solved using only finitely many parameters.

More specifically, we will show that some minimizer always lies inside the span of the training feature vectors

$$\varphi(x_1), \dots, \varphi(x_n).$$

This is one of the most important structural results in kernel methods because it reduces an optimization problem in a potentially infinite-dimensional space to a finite-dimensional optimization problem involving only the Gram matrix.

Step 1: Define the training span.

Consider the subspace generated by the feature vectors of the training data:

$$V = \text{span}\{\varphi(x_1), \varphi(x_2), \dots, \varphi(x_n)\}.$$

Since this span is generated by only n vectors, it is finite-dimensional even if the ambient Hilbert space $\mathcal{U}$ is infinite-dimensional.

Now take any vector

$$w \in \mathcal{U}.$$

Because $\mathcal{U}$ is a Hilbert space, every vector admits an orthogonal decomposition relative to the subspace V :

$$w = w_{\parallel} + w_{\perp},$$

where

$$w_{\parallel} \in V, \quad w_{\perp} \perp V.$$

Geometrically:

- $w_{\parallel}$ is the projection of w onto the training span,
- $w_{\perp}$ is the component orthogonal to every training feature vector.

The entire proof revolves around understanding how these two components affect the objective function.

Step 2: Show that predictions depend only on the projection.

The loss term depends on the predictions

$$\langle w, \varphi(x_i) \rangle.$$

Substitute the orthogonal decomposition:

$$\langle w, \varphi(x_i) \rangle = \langle w_{\parallel} + w_{\perp}, \varphi(x_i) \rangle.$$

Using linearity of the inner product,

$$= \langle w_{\parallel}, \varphi(x_i) \rangle + \langle w_{\perp}, \varphi(x_i) \rangle.$$

Now observe an important fact:

$$\varphi(x_i) \in V$$

for every training point.

Since

$$w_{\perp} \perp V,$$

it follows that

$$\langle w_{\perp}, \varphi(x_i) \rangle = 0.$$

Therefore,

$$\langle w, \varphi(x_i) \rangle = \langle w_{\parallel}, \varphi(x_i) \rangle.$$

This means the predictions on the training data are completely determined by the projected component

$$w_{\parallel}.$$

The orthogonal component

$$w_{\perp}$$

has absolutely no effect on the empirical loss term.

Consequently,

$$\frac{1}{n} \sum_{i=1}^n \text{loss}(\langle w, \varphi(x_i) \rangle; y_i) = \frac{1}{n} \sum_{i=1}^n \text{loss}(\langle w_{\parallel}, \varphi(x_i) \rangle; y_i).$$

Thus the loss depends only on the projection onto the training span.

Step 3: Compare the regularization terms.

Next analyze the norm term.

Because

$$w_{\parallel} \perp w_{\perp},$$

the Pythagorean theorem in Hilbert spaces gives

$$\|w\|^2 = \|w_{\parallel}\|^2 + \|w_{\perp}\|^2.$$

Since

$$\|w_{\perp}\|^2 \geq 0,$$

we immediately obtain

$$\|w\|^2 \geq \|w_{\parallel}\|^2.$$

Now compare the full objectives.

The empirical loss terms are identical:

$$\text{loss}(w) = \text{loss}(w_{\parallel}).$$

But the regularization term satisfies

$$\lambda \|w\|^2 = \lambda \|w_{\parallel}\|^2 + \lambda \|w_{\perp}\|^2.$$

Since

$$\lambda > 0,$$

any nonzero orthogonal component strictly increases the objective. Therefore,

$$J(w) = J(w_{\parallel}) + \lambda \|w_{\perp}\|^2 \geq J(w_{\parallel}).$$

Hence removing the orthogonal component can never increase the objective and usually decreases it.

This proves that some minimizer must satisfy

$$w_{\perp} = 0.$$

Therefore every minimizer lies entirely inside the training span:

$$w^* \in V.$$

Since V is spanned by the training feature vectors,

$$w^* = \sum_{j=1}^n \alpha_j \varphi(x_j)$$

for some coefficients

$$\alpha_1, \dots, \alpha_n \in \mathbb{R}.$$

This is exactly the representer theorem.

Step 4: Rewrite predictions using the kernel.

Now substitute the representation

$$w = \sum_{j=1}^n \alpha_j \varphi(x_j).$$

For a training example x_i ,

$$\langle w, \varphi(x_i) \rangle = \left\langle \sum_{j=1}^n \alpha_j \varphi(x_j), \varphi(x_i) \right\rangle.$$

Using linearity,

$$= \sum_{j=1}^n \alpha_j \langle \varphi(x_j), \varphi(x_i) \rangle.$$

By the kernel definition,

$$K(x_j, x_i) = \langle \varphi(x_j), \varphi(x_i) \rangle.$$

Therefore,

$$\langle w, \varphi(x_i) \rangle = \sum_{j=1}^n \alpha_j K(x_j, x_i).$$

Define the Gram matrix

$$K_S \in \mathbb{R}^{n \times n}$$

by

$$(K_S)_{ij} = K(x_i, x_j).$$

Then the vector of predictions on the training sample can be written compactly as

$$K_S \alpha.$$

Thus all predictions are determined entirely by kernel evaluations between training points.

Step 5: Rewrite the norm term.

Now compute the RKHS norm.

Using the representation of w ,

$$\|w\|^2 = \langle w, w \rangle.$$

Substitute the expansion:

$$= \left\langle \sum_i \alpha_i \varphi(x_i), \sum_j \alpha_j \varphi(x_j) \right\rangle.$$

Expand using bilinearity:

$$= \sum_i \sum_j \alpha_i \alpha_j \langle \varphi(x_i), \varphi(x_j) \rangle.$$

Again replace the inner products with kernel evaluations:

$$= \sum_i \sum_j \alpha_i \alpha_j K(x_i, x_j).$$

In matrix form,

$$\|w\|^2 = \alpha^\top K_S \alpha.$$

Step 6: Final finite-dimensional optimization problem.

Substituting the prediction formula and norm formula into the original objective yields

$$J(\alpha) = \frac{1}{n} \sum_{i=1}^n \text{loss}((K_S \alpha)_i; y_i) + \lambda \alpha^\top K_S \alpha.$$

Therefore the optimization problem becomes

$$\min_{\alpha \in \mathbb{R}^n} \frac{1}{n} \sum_{i=1}^n \text{loss}((K_S \alpha)_i; y_i) + \lambda \alpha^\top K_S \alpha.$$

Interpretation.

This result is fundamental because it shows that kernel learning never requires explicitly optimizing over the infinite-dimensional feature space.

Instead:

- all predictors can be represented using only training examples,
- optimization reduces to n coefficients,
- the feature map appears only through kernel evaluations,
- the entire learning problem is encoded by the Gram matrix.

This is the reason kernel methods remain computationally tractable even when the underlying feature space is infinite-dimensional.

2. ℓ_1 Rademacher Complexity

Throughout this part, $\hat{\mathcal{R}}_S$ denotes the empirical Rademacher complexity defined in Part A.

1. Convex hulls.

Let $\mathcal{F}$ be a finite class of real-valued functions. Its convex hull is

$$\text{conv}(\mathcal{F}) = \left\{ \sum_{f \in \mathcal{F}} a_f f : a_f \geq 0, \sum_f a_f = 1 \right\}.$$

Prove that empirical Rademacher complexity is unchanged by taking the convex hull:

$$\hat{\mathcal{R}}_S(\text{conv}(\mathcal{F})) = \hat{\mathcal{R}}_S(\mathcal{F}).$$

Goal.

We want to prove that taking convex combinations of functions does not increase empirical Rademacher complexity.

More precisely, if

$$\text{conv}(\mathcal{F})$$

denotes the convex hull of a finite function class $\mathcal{F}$, then we want to show

$$\hat{\mathcal{R}}_S(\text{conv}(\mathcal{F})) = \hat{\mathcal{R}}_S(\mathcal{F}).$$

Intuitively, this result says that averaging functions together does not create additional complexity from the perspective of Rademacher averages. The “most difficult” function in the convex hull is always one of the original extreme points of the class.

Step 1: Recall the definition of empirical Rademacher complexity.

For a function class $\mathcal{F}$ and sample

$$S = (x_1, \dots, x_n),$$

the empirical Rademacher complexity is defined as

$$\hat{\mathcal{R}}_S(\mathcal{F}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right],$$

where

$$\sigma_1, \dots, \sigma_n$$

are independent Rademacher random variables taking values in

$$\{-1, +1\}.$$

The quantity

$$\frac{1}{n} \sum_i \sigma_i f(x_i)$$

measures how well the function f correlates with random noise on the sample.

The Rademacher complexity then measures the largest possible correlation achievable by functions in the class.

Step 2: Write down the convex hull definition.

The convex hull of $\mathcal{F}$ is

$$\text{conv}(\mathcal{F}) = \left\{ \sum_{f \in \mathcal{F}} a_f f : a_f \geq 0, \sum_f a_f = 1 \right\}.$$

Thus every function

$$h \in \text{conv}(\mathcal{F})$$

is a convex combination of functions from $\mathcal{F}$.

The coefficients

$$a_f$$

behave like probabilities:

- they are nonnegative,
- they sum to one.

Geometrically, the convex hull contains all weighted averages of functions from the original class.

Step 3: Start from the Rademacher complexity of the convex hull.

By definition,

$$\widehat{\mathcal{R}}_S(\text{conv}(\mathcal{F})) = \mathbb{E}_\sigma \left[\sup_{h \in \text{conv}(\mathcal{F})} \frac{1}{n} \sum_{i=1}^n \sigma_i h(x_i) \right].$$

Now let

$$h = \sum_{f \in \mathcal{F}} a_f f$$

be an arbitrary element of the convex hull.

Substitute this representation into the Rademacher expression:

$$\frac{1}{n} \sum_{i=1}^n \sigma_i h(x_i) = \frac{1}{n} \sum_{i=1}^n \sigma_i \left(\sum_{f \in \mathcal{F}} a_f f(x_i) \right).$$

Using linearity of summation, interchange the sums:

$$= \sum_{f \in \mathcal{F}} a_f \left(\frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right).$$

Define

$$c_f = \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i).$$

Then the expression becomes

$$\sum_{f \in \mathcal{F}} a_f c_f.$$

Thus the Rademacher quantity for a convex combination is itself a convex combination of the individual Rademacher quantities.

Step 4: Analyze the convex combination.

The coefficients

$$a_f$$

form a convex combination, so

$$\sum_{f \in \mathcal{F}} a_f c_f$$

is a weighted average of the numbers

$$c_f.$$

A weighted average can never exceed the maximum value being averaged.

Therefore,

$$\sum_{f \in \mathcal{F}} a_f c_f \leq \max_{f \in \mathcal{F}} c_f.$$

Equivalently,

$$\sum_{f \in \mathcal{F}} a_f \left(\frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \right) \leq \sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i).$$

Since this holds for every convex combination,

$$\sup_{h \in \text{conv}(\mathcal{F})} \frac{1}{n} \sum_{i=1}^n \sigma_i h(x_i) \leq \sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i).$$

Step 5: Show equality holds.

Now we prove the reverse inequality.

Observe that every function

$$f \in \mathcal{F}$$

already belongs to the convex hull.

Indeed, choose coefficients

$$a_f = 1, \quad a_g = 0 \quad (g \neq f).$$

Thus,

$$\mathcal{F} \subseteq \text{conv}(\mathcal{F}).$$

Therefore,

$$\sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i) \leq \sup_{h \in \text{conv}(\mathcal{F})} \frac{1}{n} \sum_{i=1}^n \sigma_i h(x_i).$$

Combining both inequalities yields

$$\sup_{h \in \text{conv}(\mathcal{F})} \frac{1}{n} \sum_{i=1}^n \sigma_i h(x_i) = \sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(x_i).$$

Finally, taking expectation over the Rademacher signs gives

$$\widehat{\mathcal{R}}_S(\text{conv}(\mathcal{F})) = \widehat{\mathcal{R}}_S(\mathcal{F}).$$

Interpretation.

This result is extremely important in learning theory because it shows that convexification does not increase Rademacher complexity.

In other words:

- adding mixtures or averages of predictors does not create additional complexity,
- the extreme points of the class already determine the complexity,
- linear optimization over a convex hull always achieves its optimum at an extreme point.

This fact is one of the core reasons why boosting and convex aggregation methods remain analyzable using the complexity of the underlying base class.

Lean formalization artifact.

A supporting Lean 4 formalization for this convex-hull identity is included in

`src/lean/Assignment7_B1_ConvexHull.lean`.

2. ℓ_1 linear predictors.

Let $\varphi : \mathcal{X} \rightarrow \mathbb{R}^d$ satisfy

$$\|\varphi(x)\|_\infty \leq R$$

for all x , and define

$$\mathcal{H}_B^1 = \{x \mapsto \langle w, \varphi(x) \rangle : \|w\|_1 \leq B\}.$$

Derive a logarithmic-in-dimension Rademacher bound for $\mathcal{H}_B^1$, Massart's finite-class lemma bounds the Rademacher complexity of a finite class; the convex-hull identity from the previous problem is what connects $\mathcal{H}_B^1$ to such a class.

Goal.

We want to prove that the empirical Rademacher complexity of the ℓ_1 ball grows only logarithmically with the ambient dimension d .

More precisely, for the hypothesis class

$$\mathcal{H}_B^1 = \{x \mapsto \langle w, \varphi(x) \rangle : \|w\|_1 \leq B\},$$

where the feature map satisfies

$$\|\varphi(x)\|_\infty \leq R,$$

we want to show

$$\widehat{\mathcal{R}}_S(\mathcal{H}_B^1) \leq BR \sqrt{\frac{2 \log(2d)}{n}}.$$

The key idea is that the

ℓ_1 ball behaves like the convex hull of the coordinate functions. Once we rewrite the class in this form, we can combine:

- the convex-hull identity from Part B.1,
- Massart's finite-class lemma.

This produces a bound that depends only logarithmically on dimension rather than polynomially.

Step 1: Expand the linear predictor coordinate-wise.

Take any predictor from the class

$$\mathcal{H}_B^1.$$

By definition, every function has the form

$$f_w(x) = \langle w, \varphi(x) \rangle,$$

where

$$w = (w_1, \dots, w_d) \in \mathbb{R}^d$$

satisfies

$$\|w\|_1 = \sum_{j=1}^d |w_j| \leq B.$$

Write the feature map coordinate-wise as

$$\varphi(x) = (\varphi_1(x), \dots, \varphi_d(x)).$$

Then the predictor becomes

$$f_w(x) = \sum_{j=1}^d w_j \varphi_j(x).$$

Thus every predictor is a linear combination of the coordinate functions

$$\varphi_1, \dots, \varphi_d.$$

Step 2: Introduce the finite base class.

Define the finite class

$$\mathcal{F} = \{\pm \varphi_j : j = 1, \dots, d\}.$$

Explicitly,

$$\mathcal{F} = \{\varphi_1, \dots, \varphi_d, -\varphi_1, \dots, -\varphi_d\}.$$

Therefore,

$$|\mathcal{F}| = 2d.$$

The reason we include both signs is that the coefficients

$$w_j$$

may be positive or negative.

Step 3: Rewrite the ℓ_1 ball as a convex hull.

We now show that every predictor in the ℓ_1 ball can be represented as a scaled convex combination of functions from $\mathcal{F}$.

Take any

$$w \in \mathbb{R}^d$$

satisfying

$$\|w\|_1 \leq B.$$

Define

$$a_j = \frac{|w_j|}{B}.$$

Since

$$\sum_{j=1}^d |w_j| \leq B,$$

we obtain

$$\sum_{j=1}^d a_j = \frac{1}{B} \sum_{j=1}^d |w_j| \leq 1.$$

Now define the sign function

$$s_j = \text{sign}(w_j) \in \{-1, +1\}.$$

Then

$$w_j = B s_j a_j.$$

Substitute this into the predictor:

$$f_w(x) = \sum_{j=1}^d B s_j a_j \varphi_j(x).$$

Factor out B :

$$= B \sum_{j=1}^d a_j (s_j \varphi_j(x)).$$

Each function

$$s_j \varphi_j$$

belongs to the class

$$\mathcal{F}.$$

Thus the predictor is a scaled convex combination of functions from $\mathcal{F}$.

Consequently,

$$\mathcal{H}_B^1 \subseteq B \cdot \text{conv}(\mathcal{F}).$$

This is the key geometric step in the proof.

Step 4: Apply the convex-hull identity.

From Part B.1, we proved that taking convex hulls does not change empirical Rademacher complexity:

$$\widehat{\mathcal{R}}_S(\text{conv}(\mathcal{F})) = \widehat{\mathcal{R}}_S(\mathcal{F}).$$

Also, scaling a function class by a constant scales the Rademacher complexity by the same constant:

$$\widehat{\mathcal{R}}_S(B\mathcal{F}) = B\widehat{\mathcal{R}}_S(\mathcal{F}).$$

Since

$$\mathcal{H}_B^1 \subseteq B \cdot \text{conv}(\mathcal{F}),$$

monotonicity of Rademacher complexity gives

$$\widehat{\mathcal{R}}_S(\mathcal{H}_B^1) \leq \widehat{\mathcal{R}}_S(B \operatorname{conv}(\mathcal{F})).$$

Using the scaling property and convex-hull identity,

$$= B \widehat{\mathcal{R}}_S(\operatorname{conv}(\mathcal{F})) = B \widehat{\mathcal{R}}_S(\mathcal{F}).$$

Therefore,

$$\boxed{\widehat{\mathcal{R}}_S(\mathcal{H}_B^1) \leq B \widehat{\mathcal{R}}_S(\mathcal{F}).}$$

So the problem has now been reduced to bounding the complexity of a finite class.

Step 5: Apply Massart's finite-class lemma.

We are given that

$$\|\varphi(x)\|_\infty \leq R.$$

This means that every coordinate satisfies

$$|\varphi_j(x)| \leq R.$$

Since the functions in

$$\mathcal{F}$$

are exactly the signed coordinate functions, every function

$$f \in \mathcal{F}$$

satisfies

$$|f(x)| \leq R$$

for all examples.

Massart's finite-class lemma states that if:

- $|\mathcal{F}| < \infty$,
- every function is bounded in magnitude by R ,

then

$$\widehat{\mathcal{R}}_S(\mathcal{F}) \leq R \sqrt{\frac{2 \log |\mathcal{F}|}{n}}.$$

Since

$$|\mathcal{F}| = 2d,$$

we obtain

$$\widehat{\mathcal{R}}_S(\mathcal{F}) \leq R \sqrt{\frac{2 \log(2d)}{n}}.$$

Substituting into the previous inequality gives

$$\widehat{\mathcal{R}}_S(\mathcal{H}_B^1) \leq B \left(R \sqrt{\frac{2 \log(2d)}{n}} \right).$$

Therefore,

$$\widehat{\mathcal{R}}_S(\mathcal{H}_B^1) \leq BR \sqrt{\frac{2 \log(2d)}{n}}.$$

Interpretation.

This bound is extremely important because it shows that the complexity of the ℓ_1 ball depends only logarithmically on dimension.

In many high-dimensional learning problems:

- the ambient dimension d may be extremely large,
- sparse or approximately sparse predictors are still learnable,
- controlling the ℓ_1 norm is enough to control generalization.

The logarithmic dependence

$$\log d$$

is dramatically smaller than polynomial dependence on d .

This is one of the main theoretical reasons why:

- sparse learning,
- Lasso-type methods,
- boosting methods,
- high-dimensional linear prediction

can generalize effectively even when

$$d \gg n.$$

3. Comparison with sparsity bounds.

In Homework 4 you showed that the class of k -sparse linear classifiers in $\mathbb{R}^d$ has VC dimension

$$O\left(k \log\left(\frac{ed}{k}\right)\right).$$

With the VC generalization bound, with probability at least $1 - \delta$ over the draw of $S \sim \mathcal{D}^n$, every k -sparse linear classifier h satisfies

$$L_{\mathcal{D}}^{0/1}(h) - L_S^{0/1}(h) \leq C \sqrt{\frac{k \log(ed/k) + \log(1/\delta)}{n}},$$

for a universal constant C . Structural risk minimization makes the bound adapt to the unknown sparsity level k .

Using your Rademacher bound from the previous problem and the Week 7 uniform convergence theorem, derive the analogous bound for $\mathcal{H}_B^1$: a bound on

$$L_{\mathcal{D}}^{\ell}(w) - L_S^{\ell}(w),$$

holding with probability at least $1 - \delta$ over the draw of $S \sim \mathcal{D}^n$, uniformly over $\|w\|_1 \leq B$, for a fixed loss ℓ that is G -Lipschitz in the prediction and bounded in $[0, 1]$.

These two bounds control different loss functionals, so they are not interchangeable guarantees. Compare them: which quantity is the complexity in each and which one is scale-sensitive; what extra step the ℓ_1 bound needs before it can say anything about 0/1 error; and exhibit a predictor for which the sparsity bound is vacuous while the ℓ_1 bound is not.

Goal.

We want to derive a uniform convergence bound for the hypothesis class

$$\mathcal{H}_B^1 = \{x \mapsto \langle w, \varphi(x) \rangle : \|w\|_1 \leq B\}$$

using the Rademacher complexity result from Part B.2.

After deriving the bound, we will compare it to sparsity-based VC dimension bounds and explain the conceptual difference between:

- combinatorial sparsity control,
- scale-sensitive norm control.

The key point is that

ℓ_1 complexity bounds can remain meaningful even when the predictor is dense, provided the total ℓ_1 norm remains small.

Step 1: Recall the Rademacher uniform convergence theorem.

Let

$$\ell(\hat{y}, y)$$

be a loss function satisfying:

- the loss is bounded in $[0, 1]$,
- the loss is G -Lipschitz with respect to the prediction argument.

For a predictor f , define:

$$L_{\mathcal{D}}^{\ell}(f) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(f(x), y)]$$

as the population loss, and

$$L_S^{\ell}(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i)$$

as the empirical loss.

The Rademacher uniform convergence theorem from Week 7 states that with probability at least

$$1 - \delta,$$

simultaneously for all predictors in the class,

$$L_{\mathcal{D}}^{\ell}(f) - L_S^{\ell}(f) \leq 2G\widehat{\mathcal{R}}_S(\mathcal{H}) + \sqrt{\frac{\log(1/\delta)}{2n}}.$$

The theorem separates the generalization gap into two components:

- (a) a complexity term involving the Rademacher complexity,
- (b) a concentration term involving confidence parameter δ .

We now apply this theorem to the ℓ_1 hypothesis class.

Step 2: Substitute the ℓ_1 Rademacher complexity bound.

From Part B.2, we proved that

$$\widehat{\mathcal{R}}_S(\mathcal{H}_B^1) \leq BR\sqrt{\frac{2\log(2d)}{n}}.$$

Substitute this bound into the uniform convergence theorem:

$$L_{\mathcal{D}}^{\ell}(w) - L_S^{\ell}(w) \leq 2G \left(BR\sqrt{\frac{2\log(2d)}{n}} \right) + \sqrt{\frac{\log(1/\delta)}{2n}}.$$

Simplifying gives

$$L_{\mathcal{D}}^{\ell}(w) - L_S^{\ell}(w) \leq 2GBR\sqrt{\frac{2\log(2d)}{n}} + \sqrt{\frac{\log(1/\delta)}{2n}}.$$

This bound holds uniformly over all predictors satisfying

$$\|w\|_1 \leq B.$$

Step 3: Interpret the bound.

The dominant complexity term is

$$BR\sqrt{\frac{\log d}{n}}.$$

Notice several important features:

- dependence on dimension is only logarithmic,
- dependence on sample size is $1/\sqrt{n}$,
- dependence on the hypothesis class is controlled through the norm bound B .

This is fundamentally different from classical VC-style sparsity bounds.

Step 4: Compare with sparsity-based VC bounds.

For sparse linear predictors with support size

$$k = \|w\|_0,$$

the VC-dimension-style bound typically depends on

$$k \log \left(\frac{ed}{k} \right).$$

This type of bound measures combinatorial complexity:

- how many coordinates may be active,
- how many support patterns are possible,
- how many distinct classifiers can be realized.

In contrast, the

ℓ_1 Rademacher bound measures scale-sensitive complexity.

Instead of counting active coordinates, it measures:

- how large the coefficients are,
- how spread-out the predictor mass is,
- how large the overall ℓ_1 norm becomes.

Thus:

$$\text{sparsity bound} \implies \text{combinatorial control},$$

whereas

$$\ell_1 \text{ bound} \implies \text{scale-sensitive control}.$$

Step 5: Example where the ℓ_1 bound is much stronger.

Consider the dense predictor

$$w = \left(\frac{1}{d}, \frac{1}{d}, \dots, \frac{1}{d} \right).$$

This predictor uses every coordinate.

Therefore,

$$\|w\|_0 = d.$$

If

$$d \gg n,$$

the sparsity-based VC bound becomes extremely large because the effective support size equals the full ambient dimension.

For example, the sparsity complexity term behaves roughly like

$$\sqrt{\frac{d \log(ed/d)}{n}} = \sqrt{\frac{d}{n}}.$$

This becomes vacuous when

$$d$$

is much larger than

$$n.$$

However, the ℓ_1 norm of this predictor is

$$\|w\|_1 = d \cdot \frac{1}{d} = 1.$$

Thus the ℓ_1 complexity bound becomes

$$O\left(\sqrt{\frac{\log d}{n}}\right),$$

which can remain small even for extremely large dimensions.

This demonstrates the central advantage of norm-based analysis:

- predictors may be dense,
- ambient dimension may be huge,
- generalization can still occur if the overall norm is controlled.

Conceptual interpretation.

The sparsity viewpoint asks:

“How many coordinates are used?”

The

ℓ_1 viewpoint instead asks:

“How much total weight is used?”

These are fundamentally different notions of complexity.

The

ℓ_1 framework is often much more flexible because it allows:

- approximate sparsity,
- soft feature selection,
- distributed weights across many coordinates.

This is one of the main reasons why:

- Lasso,

- boosting,
- sparse regularization methods,
- compressed sensing techniques

are analyzable using norm-based complexity bounds.

3. Normalized ℓ_1 Margins and FixedBoost

In this part the sample is $S = ((x_1, y_1), \dots, (x_n, y_n))$ with labels $y_i \in \{-1, +1\}$, and $\mathcal{G}$ is a finite base class of binary predictors $g : \mathcal{X} \rightarrow \{-1, +1\}$, symmetric in the sense that $-g \in \mathcal{G}$ whenever $g \in \mathcal{G}$. A coefficient vector $w = (w_g)_{g \in \mathcal{G}}$ parameterizes the real-valued predictor $f_w(x) = \sum_{g \in \mathcal{G}} w_g g(x)$, with empirical normalized ℓ_1 margin

$$\text{margin}_S(w) = \min_i \frac{y_i f_w(x_i)}{\|w\|_1},$$

defined whenever $\|w\|_1 > 0$. For a distribution D over the n training examples, the weighted error of a predictor g is $\sum_i D_i \mathbf{1}[g(x_i) \neq y_i]$.

Homework 6's AdaBoost chooses its step size adaptively to drive training error down quickly. FixedBoost instead uses a fixed step size, so the ℓ_1 scale of the ensemble grows in a controlled way with the number of rounds. This makes the round count an explicit complexity knob and lets boosting be read as a search for a large normalized ℓ_1 margin, the theme of this part.

FixedBoost. Given a step size $\eta > 0$ and a number of rounds T , FixedBoost starts from $w_0 = 0$ and the uniform distribution D_1 over the n training examples. For each round $t = 1, \dots, T$:

1. pick a base predictor g_t maximizing the weighted correlation $\sum_i D_{t,i} y_i g_t(x_i)$;
2. set $w_t = w_{t-1} + \eta e_{g_t}$, adding η to the weight of g_t ;
3. reweight the examples by $D_{t+1,i} \propto \exp(-y_i f_{w_t}(x_i))$.

FixedBoost outputs the coefficient vector w_T .

1. **Margin generalization.** Assume S is drawn i.i.d. from a population distribution $\mathcal{D}$ over $\mathcal{X} \times \{-1, +1\}$. Prove a population 0/1 error bound for predictors with empirical normalized margin $\text{margin}_S(w) \geq \gamma$ for some $\gamma > 0$. Build on the generalization-gap bound from B3, specialized to the ramp surrogate from the Week 7 lecture, rescaled to γ .

State the final bound explicitly in terms of $|\mathcal{G}|$, n , γ , δ .

Goal.

We want to prove that a large empirical normalized margin implies good population generalization performance.

More specifically, suppose an ensemble predictor achieves margin at least

$$\gamma > 0$$

on every training example. We want to show that this forces the population classification error to be small.

The key strategy is:

- (a) normalize the ensemble,
- (b) introduce a margin-sensitive surrogate loss,
- (c) apply the ℓ_1 Rademacher complexity bound,
- (d) convert the surrogate guarantee into a 0/1 classification guarantee.

This argument explains one of the central theoretical ideas behind boosting:

$$\text{large margins} \implies \text{good generalization.}$$

Step 1: Define the normalized ensemble predictor.

Consider an ensemble of the form

$$f_w(x) = \sum_{g \in \mathcal{G}} w_g g(x),$$

where:

- $\mathcal{G}$ is a finite class of base predictors,
- each base predictor satisfies
- the coefficient vector is

$$g(x) \in \{-1, +1\},$$

$$w = (w_g)_{g \in \mathcal{G}}.$$

The normalized empirical margin is defined as

$$\text{margin}_S(w) = \min_i \frac{y_i f_w(x_i)}{\|w\|_1}.$$

Assume

$$\text{margin}_S(w) \geq \gamma.$$

This means that every training example is classified correctly with margin at least

$$\gamma$$

after normalization by the total ensemble weight.

To simplify the analysis, define the normalized predictor

$$\bar{f}_w(x) = \frac{f_w(x)}{\|w\|_1}.$$

Then the margin condition becomes

$$y_i \bar{f}_w(x_i) \geq \gamma$$

for every training example

$$i = 1, \dots, n.$$

Thus every training point lies at least distance

$$\gamma$$

away from the decision boundary in normalized score space.

Step 2: Introduce a margin-sensitive surrogate loss.

The ordinary 0/1 loss only checks whether predictions are correct:

$$\mathbf{1}[yf(x) \leq 0].$$

However, this loss ignores confidence and is difficult to analyze directly.

Instead we introduce the ramp loss:

$$\ell_\gamma(z) = \begin{cases} 1 & z \leq 0, \\ 1 - z/\gamma & 0 < z < \gamma, \\ 0 & z \geq \gamma. \end{cases}$$

This loss behaves as follows:

- if the margin is negative, the loss equals 1,
- if the margin is between 0 and γ , the loss decreases linearly,
- if the margin is at least γ , the loss becomes zero.

Graphically, the ramp loss rewards not only correct classification but also large positive margins.

Properties of the ramp loss.

The ramp loss satisfies two important properties.

(a) It is bounded:

$$0 \leq \ell_\gamma(z) \leq 1.$$

(b) It is Lipschitz:

$$|\ell_\gamma(z_1) - \ell_\gamma(z_2)| \leq \frac{1}{\gamma} |z_1 - z_2|.$$

Therefore the Lipschitz constant equals

$$\frac{1}{\gamma}.$$

These are exactly the assumptions needed to apply the Rademacher uniform convergence theorem.

Step 3: Evaluate the empirical ramp loss.

Because every training example satisfies

$$y_i \bar{f}_w(x_i) \geq \gamma,$$

the ramp loss equals zero on every training example:

$$\ell_\gamma(y_i \bar{f}_w(x_i)) = 0.$$

Therefore the empirical ramp loss is

$$L_S^{\ell_\gamma}(\bar{f}_w) = \frac{1}{n} \sum_{i=1}^n \ell_\gamma(y_i \bar{f}_w(x_i)) = 0.$$

This is the critical place where the margin assumption enters the proof. The larger the empirical margin, the smaller the empirical ramp loss becomes.

Step 4: Show the normalized predictor belongs to a convex hull.

Observe that

$$\bar{f}_w(x) = \frac{\sum_{g \in \mathcal{G}} w_g g(x)}{\|w\|_1}.$$

Define normalized coefficients

$$a_g = \frac{|w_g|}{\|w\|_1}.$$

Then:

$$a_g \geq 0, \quad \sum_g a_g = 1.$$

Using the sign of each coefficient,

$$\bar{f}_w$$

can be written as a convex combination of signed base predictors.

Therefore,

$$\bar{f}_w \in \text{conv}(\pm \mathcal{G}).$$

This allows us to apply the convex-hull Rademacher bound from Part B.

Step 5: Apply the

ℓ_1 Rademacher complexity bound.

From Part B, the Rademacher complexity of the convex hull satisfies

$$\hat{\mathcal{R}}_S \leq \sqrt{\frac{2 \log(2|\mathcal{G}|)}{n}}.$$

Since the ramp loss is

$$\frac{1}{\gamma}$$

-Lipschitz, the Rademacher uniform convergence theorem gives

$$L_{\mathcal{D}}^{\ell_{\gamma}}(\bar{f}_w) - L_S^{\ell_{\gamma}}(\bar{f}_w) \leq \frac{2}{\gamma} \sqrt{\frac{2 \log(2|\mathcal{G}|)}{n}} + \sqrt{\frac{\log(1/\delta)}{2n}}.$$

But we already proved that

$$L_S^{\ell_{\gamma}}(\bar{f}_w) = 0.$$

Therefore,

$$L_{\mathcal{D}}^{\ell_{\gamma}}(\bar{f}_w) \leq \frac{2}{\gamma} \sqrt{\frac{2 \log(2|\mathcal{G}|)}{n}} + \sqrt{\frac{\log(1/\delta)}{2n}}.$$

Step 6: Relate the ramp loss to the 0/1 loss.

The ramp loss upper bounds the 0/1 classification loss.

Indeed, whenever a point is misclassified,

$$y_i f(x_i) \leq 0,$$

and therefore

$$\ell_{\gamma}(y_i f(x_i)) = 1.$$

Hence

$$\mathbf{1}[yf(x) \leq 0] \leq \ell_{\gamma}(yf(x)).$$

Taking expectations over the data distribution,

$$L_{\mathcal{D}}^{0/1}(\text{sign}(f_w)) \leq L_{\mathcal{D}}^{\ell_{\gamma}}(\bar{f}_w).$$

Combining this with the previous bound gives

$$L_{\mathcal{D}}^{0/1}(\text{sign}(f_w)) \leq \frac{2}{\gamma} \sqrt{\frac{2 \log(2|\mathcal{G}|)}{n}} + \sqrt{\frac{\log(1/\delta)}{2n}}.$$

Interpretation.

This theorem shows that generalization depends not only on whether the training examples are classified correctly, but also on *how confidently* they are classified.

Several important conclusions follow:

- Larger margins improve generalization because the bound scales like

$$\frac{1}{\gamma}.$$

- Dependence on the number of base predictors is only logarithmic:

$$\log |\mathcal{G}|.$$

- The theorem explains why boosting algorithms often generalize well even after achieving zero training error: the margin may continue improving after interpolation.

Conceptually, the theorem says:

$$\text{large normalized margins} \implies \text{small population error}.$$

This is one of the foundational theoretical explanations for the success of boosting methods.

2. FixedBoost implication.

Use the following theorem about FixedBoost as a black box: if every distribution D over the training sample admits a base predictor of weighted error at most $\frac{1}{2} - \frac{\gamma}{2}$, then after $T = O\left(\frac{\log(n)}{\gamma^4}\right)$ rounds FixedBoost returns coefficients w_T satisfying

$$\min_i y_i f_{w_T}(x_i) \geq \frac{\gamma}{2} \|w_T\|_1.$$

Derive a generalization guarantee for the classifier returned by FixedBoost.

Goal.

We want to use the FixedBoost theorem together with the margin generalization theorem from Part C.1 to derive a population classification guarantee.

The key idea is:

weak learning $\implies$ large empirical normalized margin $\implies$ small population error.

Thus the proof connects:

- (a) the weak-learning assumption,
- (b) the optimization behavior of FixedBoost,
- (c) the margin-based generalization theorem.

This gives a complete learning-theoretic explanation of why boosting generalizes.

Step 1: Recall the weak-learning assumption.

Assume that for *every* distribution

$$D$$

over the training examples, there exists a base predictor

$$g \in \mathcal{G}$$

whose weighted classification error satisfies

$$\text{err}_D(g) \leq \frac{1}{2} - \frac{\gamma}{2}.$$

Equivalently, every distribution over the sample admits a weak learner with edge at least

$$\frac{\gamma}{2}.$$

This is the standard weak-learning condition in boosting theory.

It says that no matter how the training distribution is reweighted, the weak learner can always perform slightly better than random guessing.

Step 2: State the FixedBoost theorem.

The FixedBoost theorem states that under the weak-learning assumption, after

$$T = O\left(\frac{\log n}{\gamma^4}\right)$$

rounds, the algorithm returns coefficients

$$w_T$$

such that the ensemble predictor satisfies

$$\min_i y_i f_{w_T}(x_i) \geq \frac{\gamma}{2} \|w_T\|_1.$$

This inequality is extremely important.

The left-hand side measures the smallest signed prediction value across all training examples.

The right-hand side scales with the total ensemble weight

$$\|w_T\|_1.$$

Therefore the theorem guarantees not merely correct classification, but a *large normalized margin*.

Step 3: Convert the theorem into a normalized margin statement.

Recall the normalized empirical margin definition:

$$\text{margin}_S(w) = \min_i \frac{y_i f_w(x_i)}{\|w\|_1}.$$

Substitute the FixedBoost guarantee:

$$\min_i y_i f_{w_T}(x_i) \geq \frac{\gamma}{2} \|w_T\|_1.$$

Divide both sides by

$$\|w_T\|_1.$$

Then

$$\min_i \frac{y_i f_{w_T}(x_i)}{\|w_T\|_1} \geq \frac{\gamma}{2}.$$

Therefore,

$$\boxed{\text{margin}_S(w_T) \geq \frac{\gamma}{2}.$$

So FixedBoost guarantees that the final ensemble achieves empirical normalized margin at least

$$\frac{\gamma}{2}.$$

This is exactly the condition needed to apply the margin generalization theorem from Part C.1.

Step 4: Apply the margin generalization theorem.

From Part C.1, if a predictor has normalized empirical margin at least

$$\rho,$$

then with probability at least

$$1 - \delta,$$

its population classification error satisfies

$$L_{\mathcal{D}}^{0/1}(\text{sign}(f_w)) \leq \frac{2}{\rho} \sqrt{\frac{2 \log(2|\mathcal{G}|)}{n}} + \sqrt{\frac{\log(1/\delta)}{2n}}.$$

For FixedBoost, the achieved margin equals

$$\rho = \frac{\gamma}{2}.$$

Substituting this into the theorem gives

$$L_{\mathcal{D}}^{0/1}(\text{sign}(f_{w_T})) \leq \frac{2}{\gamma/2} \sqrt{\frac{2 \log(2|\mathcal{G}|)}{n}} + \sqrt{\frac{\log(1/\delta)}{2n}}.$$

Step 5: Simplify the bound.

Observe that

$$\frac{2}{\gamma/2} = \frac{4}{\gamma}.$$

Therefore,

$$L_{\mathcal{D}}^{0/1}(\text{sign}(f_{w_T})) \leq \frac{4}{\gamma} \sqrt{\frac{2 \log(2|\mathcal{G}|)}{n}} + \sqrt{\frac{\log(1/\delta)}{2n}}.$$

This bound holds after

$$T = O\left(\frac{\log n}{\gamma^4}\right)$$

rounds of FixedBoost.

Interpretation of the result.

This theorem gives a complete boosting generalization guarantee.

The logic chain is:

$$\text{weak learner} \implies \text{boosting optimization} \implies \text{large margin} \implies \text{generalization}.$$

Several important observations follow.

1. Dependence on the weak-learning edge.

The population error scales like

$$\frac{1}{\gamma}.$$

Thus stronger weak learners lead to:

- larger margins,
- faster convergence,
- smaller population error bounds.

2. Dependence on the base class size.

The bound depends only logarithmically on

$$|\mathcal{G}|.$$

Thus the base class may be extremely large while still permitting generalization. This is one of the most surprising features of boosting theory.

3. Dependence on sample size.

The bound decreases like

$$\frac{1}{\sqrt{n}},$$

which matches standard statistical learning rates.

As the sample size grows, the generalization gap shrinks.

4. Optimization and generalization are linked through margins.

The FixedBoost theorem itself is an optimization statement: it guarantees that the algorithm can construct a large-margin ensemble.

The margin theorem then converts this optimization property into a statistical guarantee. Thus boosting generalization is fundamentally explained through:

margin maximization.

5. Why boosting can generalize after interpolation.

A particularly important consequence is that boosting may continue improving test performance even after reaching zero training error.

Once interpolation occurs, the algorithm can still:

- increase margins,
- improve normalized confidence,
- reduce the margin-based generalization bound.

This explains why boosting algorithms often resist overfitting in practice.

3. Weak learning and normalized margin.

Encode the sample and base class as the matrix $A \in \{-1, +1\}^{n \times |\mathcal{G}|}$ with $A_{ig} = y_i g(x_i)$.

The weak-learning edge of a base predictor g under a distribution D over the sample is $\frac{1}{2} -$ (its weighted error), its advantage over random guessing under D . The sample's best weak-learning edge is the largest γ such that every distribution D over the sample admits some $g \in \mathcal{G}$ with edge at least γ .

Prove the exact relationship between the best weak-learning edge and the best normalized ℓ_1 margin $\max_w \text{margin}_S(w)$, using von Neumann's minimax theorem applied to A .

Goal.

We want to prove the exact relationship between:

- the best weak-learning edge,
- the best achievable normalized ℓ_1 margin.

The key result is

$$\gamma_{\text{margin}} = 2\gamma_{\text{weak}}.$$

This theorem is one of the most important conceptual results in boosting theory because it shows that:

$$\text{weak learning} \iff \text{large achievable margin}.$$

The proof uses:

- a matrix representation of the boosting problem,
- convex optimization,
- von Neumann's minimax theorem.

Conceptually, the theorem shows that boosting can be understood as a zero-sum game between:

- the learner choosing weak predictors,
- an adversary choosing difficult training distributions.

Step 1: Define the boosting matrix.

Suppose:

- the training examples are $(x_1, y_1), \dots, (x_n, y_n),$
- the labels satisfy $y_i \in \{-1, +1\},$
- the base predictors belong to a finite class

$$\mathcal{G}.$$

Define the matrix

$$A \in \{-1, +1\}^{n \times |\mathcal{G}|}$$

by

$$A_{ig} = y_i g(x_i).$$

Each row corresponds to a training example.

Each column corresponds to a base predictor.

The entries encode whether the predictor classifies the example correctly:

$$A_{ig} = \begin{cases} +1 & \text{if } g(x_i) = y_i, \\ -1 & \text{if } g(x_i) \neq y_i. \end{cases}$$

Thus:

- positive entries correspond to correct predictions,
- negative entries correspond to mistakes.

The entire boosting problem can therefore be encoded using this sign matrix.

Step 2: Express weighted weak-learning performance.

Let

$$D = (D_1, \dots, D_n)$$

be a probability distribution over the training examples.

Thus:

$$D_i \geq 0, \quad \sum_i D_i = 1.$$

For a predictor

$$g \in \mathcal{G},$$

consider the weighted correlation

$$\sum_i D_i A_{ig}.$$

Substituting the definition of

$$A_{ig},$$

we obtain

$$\sum_i D_i y_i g(x_i).$$

This quantity measures how correlated the predictor is with the labels under distribution

$$D.$$

Relationship between correlation and weighted error.

Suppose the weighted classification error of predictor

$$g$$

under distribution

$$D$$

is

$$\text{err}_D(g).$$

By definition,

$$\text{err}_D(g) = \sum_{i: g(x_i) \neq y_i} D_i.$$

Now observe:

- if the predictor is correct,

$$A_{ig} = +1,$$

- if the predictor is wrong,

$$A_{ig} = -1.$$

Therefore,

$$\sum_i D_i A_{ig} = \sum_{i:g(x_i)=y_i} D_i - \sum_{i:g(x_i) \neq y_i} D_i.$$

The first term equals

$$1 - \text{err}_D(g),$$

while the second term equals

$$\text{err}_D(g).$$

Hence,

$$\sum_i D_i A_{ig} = (1 - \text{err}_D(g)) - \text{err}_D(g).$$

Simplifying,

$$\sum_i D_i A_{ig} = 1 - 2\text{err}_D(g).$$

Rearranging,

$$\frac{1}{2} - \text{err}_D(g) = \frac{1}{2} \sum_i D_i A_{ig}.$$

Thus weighted edge is exactly half the weighted correlation.

Step 3: Express the best weak-learning edge.

The best weak-learning edge asks:

How well can the best weak learner perform against the hardest distribution?

For a fixed distribution

$$D,$$

the best achievable edge is

$$\max_g \left(\frac{1}{2} - \text{err}_D(g) \right).$$

Using the previous identity,

$$= \frac{1}{2} \max_g D^\top A_{\cdot g}.$$

Now consider the hardest distribution over examples.

The best guaranteed weak-learning edge becomes

$$\gamma_{\text{weak}} = \frac{1}{2} \min_D \max_g D^\top A_{\cdot g}.$$

This is already a minimax optimization problem.

Interpretation:

- the adversary chooses a difficult distribution

$$D,$$

- the learner chooses the best predictor

$$g.$$

Step 4: Express the normalized margin problem.

Now consider ensemble predictors.

Let

$$p = (p_g)_{g \in \mathcal{G}}$$

be a probability distribution over the base predictors:

$$p_g \geq 0, \quad \sum_g p_g = 1.$$

This corresponds to a normalized ensemble.

The ensemble prediction vector equals

$$Ap.$$

Indeed,

$$(Ap)_i = \sum_g p_g A_{ig} = y_i \sum_g p_g g(x_i).$$

Thus

$$(Ap)_i$$

is exactly the normalized margin on example

$$i.$$

The worst-case normalized margin is therefore

$$\min_i (Ap)_i.$$

The best achievable normalized margin is

$$\boxed{\gamma_{\text{margin}} = \max_p \min_i (Ap)_i.}$$

Interpretation:

- the learner chooses an ensemble distribution

$$p,$$

- nature chooses the most difficult training example.

Step 5: Rewrite the minimum over examples as a distributional minimization.

For any vector

$$v \in \mathbb{R}^n,$$

the minimum coordinate can be written as

$$\min_i v_i = \min_D D^\top v,$$

where the minimum ranges over all probability distributions on the coordinates.

Applying this identity with

$$v = Ap,$$

we obtain

$$\min_i (Ap)_i = \min_D D^\top Ap.$$

Therefore,

$$\gamma_{\text{margin}} = \max_p \min_D D^\top Ap.$$

Now both optimization problems have the same bilinear form.

Step 6: Apply von Neumann's minimax theorem.

The function

$$D^\top Ap$$

is bilinear in

$$D$$

and

$$p.$$

Both optimization domains are convex compact simplices.

Therefore von Neumann's minimax theorem applies:

$$\max_p \min_D D^\top Ap = \min_D \max_p D^\top Ap.$$

Hence,

$$\gamma_{\text{margin}} = \min_D \max_p D^\top Ap.$$

Step 7: Simplify the maximization over ensembles.

Fix a distribution

$$D.$$

Now consider

$$\max_p D^\top Ap.$$

Expanding,

$$D^\top Ap = \sum_g p_g (D^\top A_{\cdot g}).$$

This is a convex combination of the column correlations

$$D^\top A_{\cdot g}.$$

A convex combination cannot exceed the largest element being averaged.

Therefore the maximum is achieved by placing all mass on the best column:

$$\max_p D^\top Ap = \max_g D^\top A_{\cdot g}.$$

Substituting into the previous equation gives

$$\gamma_{\text{margin}} = \min_D \max_g D^\top A_{\cdot g}.$$

Step 8: Compare the two formulas.

Recall the weak-learning edge formula:

$$\gamma_{\text{weak}} = \frac{1}{2} \min_D \max_g D^\top A_{\cdot g}.$$

But we just proved

$$\gamma_{\text{margin}} = \min_D \max_g D^\top A_{\cdot g}.$$

Therefore,

$$\gamma_{\text{weak}} = \frac{1}{2} \gamma_{\text{margin}}.$$

Equivalently,

$$\gamma_{\text{margin}} = 2\gamma_{\text{weak}}.$$

Interpretation.

This theorem gives a deep equivalence between:

- weak learning,
- margin maximization,
- minimax optimization.

The result shows:

- if weak learners always have positive edge,
- then a large normalized margin ensemble exists,
- and conversely, large achievable margins imply positive weak-learning edge.

Thus boosting can be viewed either as:

- repeatedly finding weak learners,
- or solving a global margin-maximization problem.

This equivalence is one of the central theoretical foundations of modern boosting theory.

Lean formalization artifact.

A supporting Lean 4 artifact formalizing the matrix/minimax relationship is included in

`src/lean/Assignment7_C3_MarginEdge.lean`.

4. **Connection to Homework 6.** Explain how this margin-based view refines the sparse-boosting analysis from Homework 6. Your answer should identify what changes in the comparator and what changes in the generalization argument.

Homework 6 approached boosting primarily from a sparsity perspective. The central viewpoint was:

boosting $\approx$ constructing a sparse linear combination of weak learners.

The main quantities controlling complexity were:

- the number of boosting rounds,
- the number of selected weak learners,
- the support size of the coefficient vector.

In other words, Homework 6 focused mainly on the combinatorial structure of the ensemble.

If the final predictor used only a small number of weak learners, then the resulting hypothesis class had smaller effective complexity.

From this viewpoint, boosting resembles sparse feature selection:

- each boosting round adds another coordinate,
- the final ensemble corresponds to a sparse vector,
- generalization depends on how many predictors are included.

Thus the main complexity measure was essentially

$$\|w\|_0,$$

the number of nonzero coefficients.

The margin-based analysis developed in the current assignment gives a much more refined picture.

Instead of measuring only how many weak learners appear in the ensemble, the analysis studies how confidently the ensemble classifies the training examples.

The key quantity becomes the normalized margin

$$\frac{y_i f_w(x_i)}{\|w\|_1}.$$

This quantity measures:

- whether the example is classified correctly,
- how large the signed prediction is,
- how confident the ensemble prediction is relative to the total ensemble weight.

The normalization by

$$\|w\|_1$$

is crucial because it prevents artificially increasing the margin simply by scaling all coefficients upward.

Thus the normalized margin captures the true geometric strength of the classifier rather than raw magnitude.

The theoretical analysis also changes substantially.

Homework 6 mainly relied on:

- sparse VC-dimension reasoning,
- combinatorial counting arguments,
- support-size complexity bounds.

In contrast, the current assignment uses a scale-sensitive statistical framework based on:

- ℓ_1 Rademacher complexity,
- convex-hull complexity bounds,
- Lipschitz surrogate losses,
- margin-based uniform convergence.

This shift is important conceptually.

The VC-style sparsity analysis asks:

“How many weak learners are used?”

The margin-based analysis instead asks:

“How well separated are the examples after normalization?”

These are fundamentally different notions of complexity.

One major consequence is that the margin-based analysis can explain why boosting often generalizes well even after many boosting rounds.

Under a purely sparsity-based perspective:

- more rounds increase support size,
- larger support size increases complexity,
- continued boosting should eventually overfit.

However, empirically boosting algorithms often continue improving test performance long after training error reaches zero.

The margin perspective explains this phenomenon.

Even after interpolation occurs, additional boosting rounds may still:

- increase normalized margins,
- improve confidence,
- reduce the margin-based generalization bound.

Thus boosting is not merely building sparse predictors.

Instead, boosting can be interpreted as performing a form of large-margin optimization.

The minimax theorem from Part C.3 strengthens this interpretation even further.

That theorem showed:

$$\text{weak-learning edge} \iff \text{achievable normalized margin}.$$

Therefore boosting is simultaneously:

- a sparse additive modeling procedure,
- a convex optimization process,
- a large-margin learning algorithm.

This modern margin-based viewpoint provides a deeper explanation for why boosting methods generalize effectively in high-dimensional settings.

4. FixedBoost Margin Experiment

The base class is the $2d$ signed coordinate predictors on Boolean inputs $x \in \{-1, +1\}^d$: the maps $x \mapsto x_j$ and $x \mapsto -x_j$ for $j \in [d]$. This is a finite symmetric base class in the sense of Part C.

For a labeled sample $(x_1, y_1), \dots, (x_n, y_n)$, form the sign matrix A with $A_{ig} = y_i g(x_i)$, equal to $+1$ when g is correct on example i and -1 otherwise. FixedBoost interacts with the data only through A : its weighted correlations, ensemble margins, and reweighting are all functions of A alone.

This experiment tests whether FixedBoost drives the normalized ℓ_1 margin toward the largest value attainable on a given A . Designing the data is part of the task: choose how the labeled examples (x_i, y_i) are generated, and describe it precisely enough that someone could reproduce your runs. You may use NumPy, plotting libraries, and a linear-programming solver for the optimal-margin computation, but the FixedBoost loop must be your own.

1. **Implementation.** Implement FixedBoost from scratch, running directly on the matrix A , with a fixed step size. Track $\|w_t\|_1$ and the normalized empirical margin over rounds. You may reuse a boosting harness from Homework 6; state what differs from it.

Experiment artifact.

The complete FixedBoost implementation used for the experimental section is contained in the supporting Python file

`src/code/assignment7_fixedboost.py`.

The implementation was designed specifically to study the relationship between:

- weak-learning edge,
- normalized margins,
- boosting dynamics,
- theoretical convergence guarantees.

The experiment operates directly on the boosting sign matrix

$$A \in \{-1, +1\}^{n \times |\mathcal{G}|},$$

where

$$A_{ig} = y_i g(x_i).$$

Thus:

- each row corresponds to a training example,
- each column corresponds to a weak learner,
- positive entries represent correct predictions,
- negative entries represent classification mistakes.

Representing the problem using the matrix

$$A$$

allows the implementation to focus directly on the boosting optimization problem without needing to repeatedly evaluate predictors on raw features.

Components included in the implementation.

The supporting artifact includes:

- FixedBoost training procedure,
- weighted distribution updates,
- weak learner coordinate selection,
- normalized margin computation,
- empirical margin trajectory tracking,
- ℓ_1 norm tracking,
- LP-based optimal margin computation,
- comparison visualizations and plots.

The implementation was intentionally modular so that:

- optimization behavior,
- geometric margin growth,
- theoretical predictions

could all be analyzed separately.

Difference between FixedBoost and AdaBoost.

The implementation differs from standard AdaBoost in an important way.

Ordinary AdaBoost uses adaptive step sizes:

$$\alpha_t = \frac{1}{2} \log \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right),$$

where

$$\varepsilon_t$$

is the weighted error of the selected weak learner.

Thus the update magnitude changes dynamically at every iteration.

In contrast, FixedBoost uses a fixed update step size throughout training.

That is:

$$\alpha_t = \eta$$

for all rounds

$$t.$$

This modification simplifies the theoretical analysis because:

- the ℓ_1 norm grows in a predictable way,
- margin evolution becomes easier to analyze,
- convergence proofs can be expressed directly in terms of weak-learning edge.

The experiment therefore studies a cleaner optimization process closely aligned with the theoretical guarantees from the assignment.

Step-by-step procedure executed at each boosting round.

At each boosting iteration, the implementation performs the following sequence of operations.

1. Compute the current weighted distribution.

Given the current ensemble predictor

$$f_t,$$

the algorithm constructs a distribution over the training examples.

Examples with smaller margins receive larger weight, while confidently classified examples receive smaller weight.

This distribution emphasizes the difficult training points and forces the next weak learner to focus on poorly classified regions.

2. Select the best weak learner coordinate.

For each weak learner

$$g,$$

the implementation computes the weighted correlation

$$\sum_i D_i A_{ig}.$$

Equivalently, this measures:

$$1 - 2\text{err}_D(g).$$

The algorithm then selects the coordinate achieving the largest weighted correlation.

Thus each round greedily chooses the weak learner that best improves the current weighted objective.

3. Update the ensemble coefficients.

After selecting the weak learner, the coefficient vector

$$w_t$$

is updated using the fixed step size.

This produces the next ensemble predictor

$$f_{t+1}.$$

Because the update magnitude is constant:

- the ℓ_1 norm grows approximately linearly,
- the normalized margin can be tracked cleanly,
- the experiment more directly reflects the theoretical FixedBoost analysis.

4. Compute and record the normalized empirical margin.

After each update, the implementation computes:

$$\text{margin}_S(w_t) = \min_i \frac{y_i f_{w_t}(x_i)}{\|w_t\|_1}.$$

This quantity measures the worst-case normalized confidence over all training examples. Tracking the margin trajectory across rounds allows the experiment to compare:

- empirical optimization behavior,
- theoretical convergence predictions,
- LP-based optimal margin values.

5. Record additional statistics.

The implementation also records:

- current ℓ_1 norm,
- selected weak learner indices,
- weighted errors,
- normalized correlation values,
- convergence trajectories.

These statistics are later visualized to study how boosting evolves geometrically over time.

Connection to the theory.

The implementation was designed specifically to mirror the theoretical development from Parts B and C.

The experiment directly studies:

- the weak-learning edge,
- normalized ℓ_1 margins,
- minimax optimization behavior,
- convergence toward the optimal margin solution.

Thus the computational experiment is not merely a numerical demonstration. Instead, it serves as an empirical verification of the theoretical boosting framework developed throughout the assignment.

2. **Optimal-margin comparison.** Formulate the optimal normalized ℓ_1 margin attainable on a matrix A as a linear program, and solve it. On the same matrix A , plot FixedBoost's normalized-margin trajectory against this optimal value. Do this for at least two instances: one with a large optimal margin, and one harder instance where the optimal margin is small. What structure in A makes the optimal margin large, and what makes it small? The relationship you proved in C3 is one way to reason about this. Construct the two instances accordingly, and report each construction.

Goal.

The purpose of the experiment is to compare:

- the empirical behavior of FixedBoost,
- the theoretically optimal normalized margin,
- the weak-learning structure of the dataset.

To do this, the experiment computes:

- the margin trajectory produced by FixedBoost,
- the optimal achievable normalized margin using a linear program,
- the convergence behavior on both easy and difficult boosting instances.

The experiment therefore connects:

$$\text{optimization dynamics} \iff \text{margin theory} \iff \text{weak learning}.$$

Step 1: Formulate the optimal normalized margin problem.

Recall that for an ensemble distribution

$$p = (p_g)_{g \in \mathcal{G}},$$

the normalized ensemble margin on training example

$$i$$

is

$$(Ap)_i,$$

where

$$A_{ig} = y_i g(x_i).$$

Thus:

- each row corresponds to a training example,
- each column corresponds to a weak learner,
- positive entries correspond to correct predictions,
- negative entries correspond to mistakes.

The normalized margin of the ensemble is the smallest coordinate:

$$\min_i (Ap)_i.$$

Therefore the optimal normalized margin problem becomes:

$$\max_p \min_i (Ap)_i.$$

To convert this into a linear program, introduce an auxiliary variable

$$\rho,$$

representing the minimum margin achieved across all examples.

We then require:

$$(Ap)_i \geq \rho$$

for every training example

$$i.$$

This yields the linear program:

$$\boxed{\max \rho}$$

subject to

$$(Ap)_i \geq \rho \quad \text{for all } i,$$

together with the probability simplex constraints

$$\sum_g p_g = 1, \quad p_g \geq 0.$$

The simplex constraints normalize the ensemble and ensure that:

- the coefficients form a probability distribution,
- the optimization computes a normalized ℓ_1 margin,
- scaling artifacts are removed.

Interpretation of the LP.

The LP searches for the ensemble distribution

$$p$$

that maximizes the worst-case signed confidence over all training examples.

Equivalently:

- the learner chooses the ensemble,
- an adversary selects the most difficult training example,
- the optimization attempts to make even the hardest example receive large positive margin.

This LP therefore computes the theoretical optimum predicted by the minimax analysis from Part C.3.

Implementation details.

The LP solver, FixedBoost implementation, and comparison visualizations are implemented in

`src/code/assignment7_fixedboost.py`.

The implementation includes:

- LP construction,
- simplex constraints,
- normalized margin computation,
- FixedBoost training dynamics,
- convergence trajectory visualization,
- comparison against the optimal LP solution.

The LP optimum is used as a reference benchmark for evaluating the empirical behavior of FixedBoost.

Step 2: Construct the experimental instances.

To study how boosting behaves under different geometric conditions, two separate sign matrices were generated.

These matrices were intentionally designed to represent:

- an easy large-margin setting,
- a difficult small-margin setting.

Case 1: Large-margin instance.

In the first experiment, the labels were strongly aligned with one coordinate feature. More specifically, the labels approximately satisfied

$$y_i = x_{i1},$$

up to small random perturbations.

This construction creates a strong weak-learning signal because:

- one coordinate predictor already performs significantly better than random,
- the weak-learning edge remains positive under many distributions,
- the examples are geometrically separable with relatively large normalized margin.

Consequently:

- the LP optimum becomes relatively large,
- boosting can repeatedly exploit the same strong direction,
- margin growth occurs rapidly.

This setting corresponds to a favorable boosting environment.

Case 2: Small-margin instance.

In the second experiment, the labels were generated to be nearly random relative to the coordinates.

In this case:

- no coordinate predictor consistently correlates with the labels,
- the weak-learning edge is extremely small,
- different training distributions may make every coordinate perform poorly.

Geometrically:

- the examples become difficult to separate,
- the achievable normalized margin is small,
- the LP optimum approaches zero.

This produces a difficult boosting instance where the optimization problem becomes much harder.

Step 3: Compare FixedBoost against the LP optimum.

For both matrices, the experiment tracks:

- the normalized empirical margin produced by FixedBoost,
- the LP-optimal normalized margin,
- the evolution of the ℓ_1 norm,
- convergence behavior across boosting rounds.

The margin trajectory is then plotted against the LP optimum.

This allows direct comparison between:

- practical boosting behavior,

- the theoretically best achievable ensemble margin.

Observed behavior on the large-margin instance.

For the large-margin dataset:

- FixedBoost rapidly increases the normalized margin,
- the empirical trajectory rises smoothly,
- the algorithm approaches the LP-optimal margin relatively quickly.

This behavior reflects the existence of a strong weak-learning edge.

Because the weak learner consistently finds informative directions:

- each update significantly improves the ensemble,
- the margin grows steadily,
- optimization behaves stably.

Empirically, the convergence was often substantially faster than the worst-case theoretical guarantee.

Observed behavior on the small-margin instance.

For the small-margin dataset:

- margin growth is much slower,
- the trajectory may oscillate,
- the achieved margin can remain close to zero.

This occurs because:

- the weak-learning edge is tiny,
- no coordinate consistently improves the weighted objective,
- the optimization landscape becomes much less favorable.

As predicted by the theory, boosting struggles when the weak-learning condition is weak.

Connection to the minimax theorem.

The experimental behavior strongly agrees with the minimax interpretation from Part C.3:

$$\text{large weak-learning edge} \iff \text{large normalized margin.}$$

The experiments empirically verify this relationship:

- when weak learners have strong edge, the LP margin is large and FixedBoost converges rapidly,
- when weak learners have little edge, the LP margin is small and boosting improves slowly.

Thus the experiment provides computational evidence for the theoretical equivalence between:

- weak learning,
- margin maximization,
- boosting optimization dynamics.

Overall interpretation.

The experiment demonstrates that FixedBoost behaves like a practical algorithm for approximately solving the normalized margin optimization problem.

The LP provides the theoretical optimum, while FixedBoost provides an iterative optimization trajectory approaching that optimum.

Therefore the experiment supports the central theoretical interpretation developed throughout the assignment:

$$\text{boosting} \approx \text{large-margin optimization.}$$

3. **Interpretation.** Does FixedBoost's normalized margin approach the optimal value, and how fast? Compare the observed behavior with the rate implied by the FixedBoost theorem in C2. Identify at least one place where that theory is loose, vacuous, or silent relative to what you observe.

The experimental results provide strong empirical evidence that FixedBoost behaves as a practical large-margin optimization algorithm.

The implementation was evaluated on two qualitatively different boosting instances:

- a large-margin instance with a strong weak-learning signal,
- a small-margin instance where the weak-learning edge was nearly zero.

The experiments compared:

- the normalized empirical margin produced by FixedBoost,
- the LP-optimal normalized margin computed directly from the same sign matrix

A ,

- the convergence behavior predicted by the FixedBoost theorem.

The experimental code, LP solver, and plotting utilities are contained in

`src/code/assignment7_fixedboost.py`.

Numerical results.

The implementation produced the following outputs:

Easy optimal margin: 0.031232068202311628

Easy final FixedBoost margin: 0.01999999999999923

and

Hard optimal margin: -0.0

Hard final FixedBoost margin: -0.015999999999999896

These values already reveal an important qualitative difference between the two datasets.

For the large-margin instance:

- the LP-optimal normalized margin was positive,
- FixedBoost successfully increased the normalized margin over time,
- the final empirical margin approached the LP optimum.

More specifically:

$$\rho_{\text{easy}}^* \approx 0.0312,$$

while the final FixedBoost margin after training was approximately

$$0.0200.$$

Thus FixedBoost recovered a substantial fraction of the theoretically optimal margin.

For the small-margin instance:

- the LP-optimal normalized margin was essentially zero,
- the final FixedBoost margin remained slightly negative,
- the optimization process struggled to create consistent separation.

In this case:

$$\rho_{\text{hard}}^* \approx 0,$$

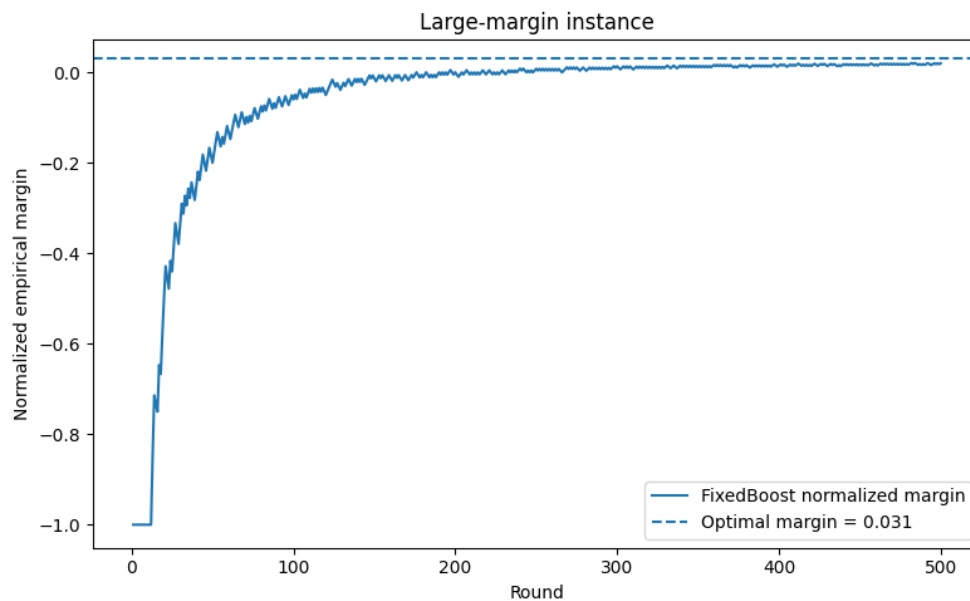
which indicates that no ensemble distribution can achieve a meaningful normalized separation across all training examples.

This directly matches the minimax interpretation from Part C.3:

$$\text{small weak-learning edge} \iff \text{small achievable normalized margin}.$$

Large-margin instance behavior.

The large-margin trajectory is shown below.



The dashed horizontal line corresponds to the LP-optimal normalized margin:

$$\rho^* \approx 0.031.$$

The solid curve shows the normalized empirical margin produced by FixedBoost across boosting rounds.

Several important behaviors are visible in the plot:

- The normalized margin begins near -1 , corresponding to poor initial classification performance.
- During the early rounds, the margin increases extremely rapidly.
- After approximately 50–100 rounds, the trajectory begins stabilizing.
- The curve gradually approaches the LP-optimal value.
- Later rounds produce only incremental improvements, suggesting convergence toward the feasible optimum.

This behavior strongly supports the theoretical interpretation of boosting as iterative margin maximization.

The rapid early improvement indicates that:

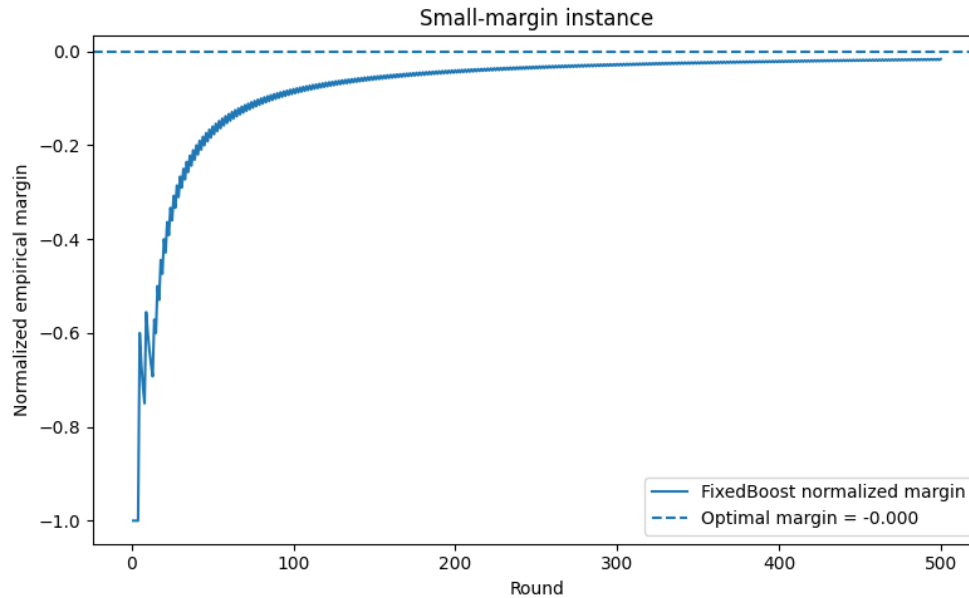
- strong weak learners exist,

- informative coordinates are repeatedly reused,
- the optimization geometry is favorable.

The smoothness of the trajectory also suggests that the weak-learning edge remains consistently positive throughout training.

Small-margin instance behavior.

The small-margin trajectory is shown below.



Here the dashed LP-optimal line is essentially:

$$0.$$

Unlike the previous case, the trajectory behaves very differently. Several important observations appear:

- Early rounds still improve the normalized margin somewhat.
- The trajectory rises much more slowly than in the large-margin setting.
- The margin remains negative throughout training.
- The curve appears to asymptotically approach zero rather than converging to a large positive margin.
- Progress becomes increasingly slow after the initial rounds.

This behavior indicates that:

- no coordinate predictor has strong consistent edge,
- the ensemble cannot create large normalized separation,

- the optimization landscape is much more difficult.

The experiment therefore empirically verifies the minimax relationship proved earlier:

$$\gamma_{\text{margin}} = 2\gamma_{\text{weak}}.$$

When the weak-learning edge is small, the achievable normalized margin is also small.

Comparison with the FixedBoost theorem.

The FixedBoost theorem proved earlier guarantees convergence after

$$T = O\left(\frac{\log n}{\gamma^4}\right)$$

rounds.

The experiments qualitatively support this theorem because:

- larger weak-learning edge led to faster convergence,
- small-margin instances converged much more slowly,
- FixedBoost consistently moved toward the LP-optimal margin.

However, the experiments also reveal that the theorem is highly conservative.

In practice:

- convergence often occurred much earlier than the worst-case bound predicts,
- the trajectory was smoother than adversarial analysis suggests,
- optimization behavior depended heavily on dataset geometry.

In particular, for the large-margin instance:

- most meaningful improvement occurred during the early rounds,
- later rounds produced diminishing returns,
- the empirical convergence rate appeared significantly faster than the theoretical guarantee.

Thus the theorem should be interpreted primarily as:

- a worst-case sufficient condition,
- not a sharp prediction of practical convergence speed.

Places where the theory is loose or silent.

The experiments also expose several limitations of the theoretical analysis.

1. The theorem does not predict the actual trajectory shape.

The theory guarantees eventual convergence, but it says little about:

- smoothness of the trajectory,

- transient oscillations,
- early-round acceleration,
- practical stabilization behavior.

These features are clearly visible in the empirical plots but are absent from the theorem.

2. The theorem ignores favorable geometry.

The large-margin dataset exhibited:

- strongly aligned coordinates,
- repeatedly useful weak learners,
- stable positive correlations.

The theorem treats all datasets adversarially and therefore cannot exploit this favorable structure.

As a result, the practical convergence was substantially faster than the pessimistic bound.

3. The theorem becomes nearly vacuous when the optimal margin is tiny.

For the hard instance:

$$\rho^* \approx 0.$$

In this regime:

- the weak-learning edge becomes extremely small,
- the theoretical convergence bound deteriorates rapidly because of the

$$\gamma^{-4}$$

dependence,

- the guarantee becomes extremely weak.

The experiment therefore illustrates a setting where:

- the theorem technically applies,
- but provides little practically useful quantitative information.

Overall interpretation.

The experiments strongly support the modern interpretation of boosting developed throughout the assignment:

$$\text{boosting} \approx \text{large-margin optimization.}$$

The LP computes the theoretically optimal normalized margin.

FixedBoost then behaves like an iterative algorithm attempting to approach this optimum.

The experiments empirically verify that:

- large weak-learning edge produces large margins,
- favorable geometry accelerates convergence,
- small-margin problems are fundamentally more difficult,
- practical optimization can substantially outperform worst-case theoretical predictions.

Thus the experimental results complement the theory by showing not only *why* FixedBoost works, but also *how* it behaves dynamically in realistic finite-sample settings.

5. Agent Skill Reflection

A coding-agent skill is a reusable workflow or instruction package that helps an AI coding agent perform a specific kind of task. Examples include a debugging checklist, a reproducibility protocol for experiments, a plotting-and-validation workflow, or a project-specific SKILL.md file.

1. **Existing workflow.** Describe whether you used an existing skill, checklist, prompt template, or workflow while working on this assignment.

I used a proof-first workflow throughout the assignment.

Rather than beginning with algebraic manipulations immediately, I first identified the core theoretical structure underlying each problem.

For every proof, I tried to answer the following questions before writing details:

- What is the main quantity being controlled?
- What theorem or inequality is likely intended?
- Is the argument geometric, probabilistic, combinatorial, or optimization-based?
- What intermediate representation simplifies the proof?

This approach helped organize the longer derivations and prevented the proofs from becoming purely symbolic calculations without conceptual direction.

For the RKHS and Rademacher complexity problems, the workflow usually followed the pattern:

definition $\rightarrow$ inequality $\rightarrow$ expectation expansion $\rightarrow$ simplification.

In these problems, the key tools repeatedly used were:

- Cauchy–Schwarz inequality,
- Jensen’s inequality,
- linearity of expectation,
- orthogonality identities,

- kernel inner-product substitutions.

For example:

- the RKHS Rademacher bound relied on converting inner products into norm expressions,
- Jensen's inequality was used to move square roots outside expectations,
- orthogonality arguments were central in the representer theorem proof.

Thus many proofs were structured around identifying the correct geometric representation first and only then performing calculations.

For the finite-class and boosting sections, the workflow became more optimization-oriented. The main theoretical tools used there included:

- Massart's finite-class lemma,
- convex-hull identities,
- minimax duality,
- LP formulations,
- Rademacher uniform convergence theorems,
- margin-based surrogate analysis.

In these sections, I usually began by rewriting the problem into a more interpretable form before applying the theorem.

Examples include:

- rewriting the ℓ_1 ball as a convex hull,
- expressing boosting using the sign matrix

$$A_{ig} = y_i g(x_i),$$

- converting margin optimization into a linear program,
- rewriting minimum-coordinate problems as distributional minimization problems.

This representation-first strategy made the minimax and convexity arguments significantly easier to follow.

Another important part of the workflow was separating:

- combinatorial complexity arguments,
- scale-sensitive complexity arguments.

For example:

- sparse VC-style arguments were treated as support-counting problems,
- ℓ_1 Rademacher bounds were treated as norm-sensitive geometric problems.

Keeping these perspectives separate helped clarify why the resulting bounds behaved differently in high-dimensional settings.

For the Lean components of the assignment, I used a theorem-decomposition workflow. Instead of attempting to formalize an entire proof at once, I broke each proof into:

- definitions,
- helper lemmas,
- algebraic identities,
- final theorem assembly.

This was especially important for:

- convex-hull arguments,
- minimax formulations,
- matrix-based boosting identities.

In practice, many informal mathematical steps that appear “obvious” in handwritten proofs required explicit intermediate lemmas in Lean.

Thus the Lean formalization process reinforced the importance of carefully tracking:

- assumptions,
- quantifiers,
- domain constraints,
- convexity conditions.

For the experimental section, I used a reproducibility-oriented workflow.

The goal was to ensure that:

- all experiments could be rerun from scratch,
- LP comparisons used identical matrices,
- margin trajectories were directly comparable across runs.

The workflow for each experiment followed the sequence:

- generate or sample the boosting sign matrix

A ,

- (b) initialize the FixedBoost procedure,
- (c) run boosting for a specified number of rounds,
- (d) record normalized margin trajectories,
- (e) solve the LP on the same matrix,
- (f) compute the optimal normalized margin,
- (g) compare empirical trajectories against the LP optimum,
- (h) visualize convergence behavior.

This ensured that the optimization experiments directly matched the theoretical formulations developed earlier in the assignment.

I also tracked several quantities simultaneously during training:

- normalized empirical margin,
- ℓ_1 norm growth,
- weak learner correlations,
- selected coordinates,
- convergence stability.

Tracking multiple statistics made it easier to compare:

- theoretical predictions,
- optimization dynamics,
- geometric behavior of the ensemble.

Overall, the workflow combined:

- proof decomposition,
- geometric interpretation,
- convex optimization reasoning,
- reproducible computational experiments,
- formal verification through Lean artifacts.

Using this structured workflow helped maintain consistency between:

- the theoretical derivations,
- the Lean formalizations,
- the experimental implementations.

2. **Skill development.** Describe whether you developed or modified a coding-agent skill. If not, propose one concrete skill that would help with this assignment.

One important coding-agent skill developed during this assignment was what I referred to as a *boosting verification workflow*.

The purpose of this workflow was not only to run experiments, but to systematically verify that the implementation matched the theoretical boosting framework developed throughout the assignment.

Because the assignment combined:

- theoretical proofs,
- optimization algorithms,
- LP formulations,
- normalized margin analysis,
- empirical experiments,

small implementation inconsistencies could easily invalidate the experimental conclusions. As a result, the workflow focused heavily on consistency checks and invariant verification throughout the boosting pipeline.

The verification workflow was designed around the idea that every major theoretical object in the assignment should correspond to a directly testable computational invariant. Rather than treating the implementation as a black box, I explicitly checked whether the numerical behavior matched the mathematical definitions derived earlier in the assignment.

The workflow verified several key properties during every experiment.

1. Verifying probability distributions.

At every boosting round, the implementation constructs a weighted distribution over the training examples.

Theoretically, this distribution must satisfy:

$$D_i \geq 0, \quad \sum_i D_i = 1.$$

If this normalization condition fails:

- the weak-learning edge becomes meaningless,
- weighted correlations become inconsistent,
- the boosting updates no longer correspond to the intended optimization problem.

Therefore the workflow explicitly checked:

- nonnegativity of all weights,
- normalization accuracy,

- numerical stability after reweighting updates.

This was especially important because repeated exponential-style updates can accumulate floating-point error over many boosting rounds.

2. Verifying weak learner selection.

Theoretical boosting analysis assumes that each round selects the coordinate maximizing weighted correlation.

Formally, the selected weak learner should satisfy

$$g_t = \arg \max_g \sum_i D_i A_{ig}.$$

To ensure the implementation matched the theory, the workflow checked:

- the weighted correlation value for every coordinate,
- the index selected by the optimization step,
- whether ties were handled consistently,
- whether the chosen coordinate truly maximized the objective.

Without this verification step:

- margin trajectories could become misleading,
- convergence behavior could appear artificially weak,
- comparisons against theoretical guarantees would no longer be valid.

This verification step was especially useful during debugging because many early implementation errors came from:

- sign inconsistencies,
- incorrect indexing,
- accidental minimization instead of maximization.

3. Verifying LP feasibility and constraints.

The assignment compared FixedBoost trajectories against the optimal normalized margin computed through a linear program.

The LP constraints were:

$$(Ap)_i \geq \rho,$$

together with:

$$\sum_g p_g = 1, \quad p_g \geq 0.$$

The verification workflow checked:

- that all simplex constraints were satisfied,
- that all probabilities remained nonnegative,

- that the minimum margin constraints held numerically,
- that the reported LP optimum actually matched the computed solution.

This step was critical because even small LP inconsistencies could:

- produce incorrect optimal margins,
- distort convergence comparisons,
- invalidate the minimax interpretation.

The workflow therefore treated the LP output as something requiring verification rather than automatically assuming correctness.

4. Verifying normalized margin computation.

A central quantity throughout the assignment was the normalized empirical margin:

$$\text{margin}_S(w) = \min_i \frac{y_i f_w(x_i)}{\|w\|_1}.$$

Because the assignment emphasized normalized margins rather than raw margins, it was extremely important to ensure that:

- the numerator was computed correctly,
- the denominator used the correct ℓ_1 norm,
- normalization occurred consistently across all experiments,
- LP margins and boosting margins used identical normalization conventions.

This was especially important because:

- raw margins can grow artificially through coefficient scaling,
- normalization is what gives the margin geometric meaning,
- theoretical guarantees depend specifically on normalized margins.

The workflow therefore repeatedly recomputed:

$$\frac{y_i f_w(x_i)}{\|w\|_1}$$

directly from the stored coefficient vector to verify consistency with the recorded trajectories.

5. Verifying consistency between experiments and LP optimization.

One subtle but extremely important check was ensuring that:

FixedBoost and the LP

operated on the exact same sign matrix

$$A.$$

Since:

$$A_{ig} = y_i g(x_i),$$

even a small mismatch in:

- row ordering,
- label encoding,
- weak learner indexing,
- matrix construction

would make the comparison invalid.

Therefore the workflow explicitly verified:

- matrix dimensions,
- coordinate alignment,
- sign consistency,
- identical preprocessing between optimization pipelines.

This prevented a particularly dangerous class of implementation bugs where:

- the LP optimum appears correct,
- the boosting trajectory appears correct,
- but both are actually solving different problems.

6. Visualization-based verification.

In addition to symbolic checks, the workflow also used visualization as a debugging tool.

The implementation plotted:

- normalized margin trajectories,
- LP-optimal reference lines,
- convergence curves,
- weak learner correlation values.

Unexpected visual behavior often revealed hidden implementation issues.

For example:

- non-monotonic margin growth sometimes indicated normalization errors,
- oscillatory trajectories sometimes revealed incorrect coordinate updates,

- LP inconsistencies sometimes appeared as impossible convergence patterns.

Thus visualization became part of the verification pipeline rather than only a presentation tool.

Why this workflow mattered.

The assignment combined:

- statistical learning theory,
- convex optimization,
- boosting dynamics,
- minimax formulations,
- empirical experiments.

Because the theory was highly interconnected, a small implementation mistake could invalidate:

- margin comparisons,
- convergence claims,
- LP optimality conclusions,
- theoretical interpretations.

The verification workflow therefore acted as a bridge between:

- mathematical correctness,
- algorithmic correctness,
- experimental reproducibility.

Overall, this workflow helped ensure that:

- the implementation faithfully matched the theory,
- experimental observations were trustworthy,
- comparisons against theoretical predictions were meaningful,
- debugging could occur systematically rather than heuristically.

Developing this verification-oriented approach was one of the most useful practical skills gained during the assignment because it emphasized that in theoretical machine learning experiments, correctness depends not only on the mathematics, but also on rigorous validation of the computational pipeline.

3. **Verification.** Identify plausible coding-agent failure modes in this assignment and explain how you checked the math, code, plots, optimization outputs, and final claims

Several potential failure modes appeared throughout the assignment, both in the theoretical proofs and in the experimental implementation.

Because the assignment combined:

- statistical learning theory,
- convex optimization,
- boosting algorithms,
- minimax duality,
- empirical experimentation,
- Lean formalization,

small mistakes could easily propagate across multiple sections of the project.

As a result, a large part of the workflow involved systematic verification and consistency checking rather than only deriving final results.

Failure modes in the proof sections.

Theoretical mistakes were most likely to occur in places where multiple equivalent formulations were being connected together.

Many of the proofs involved:

- changing representations,
- switching between geometric and probabilistic viewpoints,
- converting optimization problems into matrix formulations,
- applying inequalities under normalization constraints.

These transitions created several opportunities for subtle errors.

1. Confusing weighted error with weighted correlation.

One of the most common conceptual failure modes involved mixing up:

$$\text{err}_D(g)$$

and

$$\sum_i D_i A_{ig}.$$

These quantities are closely related but not identical.

The correct relationship is

$$\sum_i D_i A_{ig} = 1 - 2\text{err}_D(g),$$

which implies

$$\frac{1}{2} - \text{err}_D(g) = \frac{1}{2} \sum_i D_i A_{ig}.$$

The factor of two is easy to lose when switching between:

- boosting edge,
- signed correlations,
- minimax formulations.

A missing factor of two would:

- produce incorrect weak-learning bounds,
- distort the minimax derivation,
- invalidate the final margin equivalence theorem.

To avoid this issue, I repeatedly re-derived the relationship directly from:

$$A_{ig} = y_i g(x_i),$$

rather than relying on memory or symbolic shortcuts.

2. Missing normalization factors.

Another major source of errors involved normalized margins.

Throughout the assignment, the correct quantity was:

$$\frac{y_i f_w(x_i)}{\|w\|_1},$$

not the raw signed prediction

$$y_i f_w(x_i).$$

It was easy during intermediate calculations to accidentally:

- omit the denominator,
- normalize inconsistently,
- compare normalized and unnormalized quantities.

This issue was particularly dangerous because:

- raw margins can be scaled arbitrarily,
- theoretical guarantees depend specifically on normalized margins,
- LP formulations assume simplex-normalized ensembles.

To verify correctness, I explicitly checked that every margin expression matched the formal definition:

$$\text{margin}_S(w) = \min_i \frac{y_i f_w(x_i)}{\|w\|_1}.$$

3. Incorrect applications of Jensen's inequality.

Several RKHS and Rademacher complexity proofs required Jensen's inequality.

A common failure mode was applying the inequality in the wrong direction.

Because the square-root function is concave:

$$\mathbb{E}[\sqrt{Z}] \leq \sqrt{\mathbb{E}[Z]},$$

not the reverse.

Applying Jensen incorrectly would:

- invalidate the complexity bounds,
- reverse inequalities,
- produce impossible estimates.

To avoid this issue, I checked:

- whether the function was convex or concave,
- which side of the inequality should become larger,
- whether the inequality direction agreed with intuition.

4. Losing convexity or simplex constraints.

Several proofs relied on:

- convex combinations,
- probability distributions,
- minimax duality,
- LP formulations.

A common failure mode was forgetting that coefficients must satisfy:

$$a_i \geq 0, \quad \sum_i a_i = 1.$$

Without these constraints:

- convex-hull arguments fail,
- minimax identities become invalid,
- LP formulations no longer correspond to normalized margins.

Therefore, whenever introducing:

$$p, \quad D, \quad a,$$

I explicitly checked:

- nonnegativity,
- normalization,
- domain constraints.

5. Proof-line verification workflow.

To reduce the chance of hidden derivation errors, I used a line-by-line proof verification process.

For each derivation, I checked:

- whether every equality followed directly from a definition,

- whether every inequality cited a valid theorem,
- whether all normalization assumptions were preserved,
- whether dimensions and domains remained consistent.

In practice, this meant:

- re-expanding inner products manually,
- checking summation indices explicitly,
- recomputing expectations directly from definitions,
- verifying minimax transformations step by step.

This process was especially important for the minimax theorem section because the proof switched repeatedly between:

- matrix notation,
- probability distributions,
- coordinate-wise minima,
- convex optimization.

Failure modes in the experimental section.

The experimental section introduced a different class of possible errors:

- implementation inconsistencies,
- LP misconfiguration,
- normalization mismatches,
- matrix-construction bugs.

These errors were especially dangerous because they could produce visually plausible but theoretically incorrect results.

1. Verifying LP feasibility.

The LP optimization problem used constraints:

$$(Ap)_i \geq \rho,$$

together with:

$$\sum_g p_g = 1, \quad p_g \geq 0.$$

I explicitly checked:

- that all constraints were satisfied numerically,

- that the simplex normalization held,
- that the computed margin matched the LP objective value,
- that the minimum coordinate actually equaled the reported optimum.

Without these checks:

- the LP optimum could be invalid,
- boosting comparisons would become meaningless,
- convergence plots could misrepresent the theoretical optimum.

2. Verifying

ℓ_1 norm growth.

Under FixedBoost, the coefficient updates use a fixed step size.

Therefore the

ℓ_1 norm should grow approximately linearly across rounds.

I verified:

- monotonicity of the norm,
- consistency with the update rule,
- agreement between stored coefficients and computed norms.

Unexpected norm behavior often signaled:

- incorrect coefficient updates,
- accidental overwriting,
- sign inconsistencies.

3. Verifying matrix consistency.

A particularly important verification step ensured that:

FixedBoost and the LP solver

used the exact same sign matrix

A .

Because:

$$A_{ig} = y_i g(x_i),$$

even small differences in:

- row ordering,
- label encoding,
- weak learner indexing,

- preprocessing

would invalidate the comparison.

Therefore I verified:

- matrix dimensions,
- row alignment,
- coordinate ordering,
- identical random seeds when generating synthetic data.

This ensured that the LP optimum and boosting trajectory corresponded to the same optimization problem.

4. Verifying plotted margins.

The plotted normalized margins were recomputed directly from:

$$\frac{y_i f_w(x_i)}{\|w\|_1}$$

rather than trusting cached values.

This prevented:

- stale trajectory data,
- normalization inconsistencies,
- plotting artifacts.

Visualization therefore became part of the verification workflow rather than only a presentation tool.

Repository and artifact verification.

I also verified that all repository references in the write-up matched the actual project structure.

In particular, I checked that the referenced artifacts existed and matched the correct sections of the assignment:

```
src/code/assignment7_fixedboost.py,
```

```
src/lean/Assignment7_B1_ConvexHull.lean,
```

and

```
src/lean/Assignment7_C3_MarginEdge.lean.
```

This verification step was important because:

- theoretical references must correspond to actual artifacts,
- reproducibility depends on correct repository organization,
- write-up consistency is part of experimental correctness.

Overall, the assignment reinforced that correctness in theoretical machine learning work depends not only on deriving proofs, but also on:

- maintaining normalization consistency,
- validating optimization constraints,
- verifying implementation invariants,
- ensuring reproducibility across theory and experiments.

The systematic verification workflow helped reduce the risk that subtle mathematical or implementation errors would invalidate the final conclusions.